

BoxSim [BoxSimilarity] ALGORITHM NOTES_2

1. Embedding vectors, for example, generated from text examples
 - OpenAI text-embedding-3 small model with 1,536 dimensions
 - Chroma all-MiniLM-L6-v2 model with 384 dimensions
2. (Optional) Normalize each vector so its length becomes 1.
3. Convert every vector into a **binary** vector:
 - if a value $\leq 0 \rightarrow$ write **0**
 - if a value $> 0 \rightarrow$ write **1**
(you can use a different threshold instead of 0)
4. Now you have a table:
 - each **row** = one binary vector
 - each **column** = value of one element in that vector
5. Find which columns are the most “useful”:
 - a column is more useful if it has almost all 0s or almost all 1s (low variance)
 - a column is less useful if it has a mix of 0s and 1s (high variance)
 - Reorder the columns so the **most useful columns come first**.
6. Sort all rows by this binary code so similar rows come next to each other.
 - NOT necessary to sort the whole matrix, but helpful in debugging
 - Sorting ONLY up to the 63 most useful cols is needed.
 - To find the unique elements and number of each, the box counting algorithm (Liebovitch & Toth 1989) sorts elements and then counts when elements change.
 - This is the same method used by the compiled NUMPY `np.unique()`.
7. Choose a number k (called ibox):
 - look only at the **first k bits** of each row
 - if two rows have the same first k bits, put them in the **same box (cluster)**
 - Count how many rows fall into each box.
 - Increase k (1, 2, 3, ...):
 - small k $\rightarrow$ fewer, bigger clusters
 - bigger k $\rightarrow$ more, smaller clusters
9. Final output: a list of boxes/clusters for each k, with the members and counts

PROGRAM hdim_14.py codes THE HIGH DIMENSIONAL CLUSTERING ALGORITHM
for example, for clustering embedding vectors created from text samples

VARIABLES

np.array adata floating point input data

np.array bdata integer 0,1

np.array cdata cols reordered so most useful cols, lowest variance, to the left

PROGRAM lines 20-62, 145-156:

1. data input: rows separated by TAB, col separated by RETURN, no headers, no row names
2. rows = each vector, cols = elements of each vector
3. input numbers: floating point, typically [-1,1]

(I know, everyone else uses Pandas, when I learned python it was fun to create my own file opening/closing (def fliein) and parsing (def getxn) functions, I've stuck with that, sorry.)

PROGRAM lines 157-164

4. floating point to integer, convert [≤ 0 , > 0] to [0,1]

PROGRAM lines 166-185

5. find the variance = min Hamming distance to all 0's or all 1's in each COL
6. cdata = sort the cols from low to high variance

PROGRAM lines 187-215

7. not needed, but helpful in debugging
8. if you take this section out, change line 222 to: edata = cdata[:, 0:bigloop]

PROGRAM lines 217-228

9. look only at the first k bits of each row
 - if two rows have the same first k bits, put them in the same box (cluster)
 - find unique rows, how often each, use: np.unique(rows, return_counts=True)
 - Increase k (1, 2, 3, . . .) max = 63, (range(6) < 63 in line 220)
 - (beyond 63 requires np.lexsort which is much slower than np.argsort)
 - small k $\rightarrow$ fewer, bigger clusters
 - bigger k $\rightarrow$ more, smaller clusters

PROGRAM lines 229-236

10. for each k
 - output table of results, output box plot with number of rows in each box

```

1 # DEF FU7CTIONS FOR HIDIM
2 # hidim_14.py
3 # starting to add some calculation pieces
4 # FOR THIS PROGRAM USE DATA5.TXT and the smaller DATA6.txt
5 # Doing the SORT and COUNT aa GEMINI
6 # NOW trying the MASKING, for real FIRST TIME
7 # ADDING THE LAST PIECE (of this set) THE BIGLOOP
8 #COMPUTATIONS similar to hdim_13 but a few info added to TITLE
9
10 import numpy as np
11 from scipy import stats
12 import math as math
13 import matplotlib.pyplot as plt
14 import time
15 from datetime import datetime
16 import matplotlib.pyplot as plt
17 import random
18
19
20 def filein(fname):
21     numlines = 0
22     xin = []
23     f = open(fname, 'r')
24     for line in f:
25         # print (line, end="")
26         xin.append(line)
27         numlines = numlines+1
28     f.close()
29     return xin, numlines
30 data, numlines = filein(fname)
31 # print ('\ndata\n',data,'\nlines',numlines)
32 x = ['0' for i in range(numlines)]
33 for i in range(numlines):
34     x[i] = data[i].replace('\n', '')
35     x[i] = eval(x[i])
36     return x, numlines
37
38 # reads any text table with n columns
39 # elements in a row separated by a TAB
40 # rows separated by a RETURN
41
42
43 def getxn(fname):
44     data, numlines = filein(fname)
45     dataline = ['0' for i in range(numlines)]

```

```

46 for i in range(numlines):
47     x = data[i]
48     y = x.split('\t')
49     y[-1] = y[-1].replace('\n', '')
50     dataline[i] = y
51 # print ('\n\nascii-input',dataline)
52 xdata = dataline[:]
53 for i in range(numlines):
54     inline = len(dataline[i])
55     for j in range(inline):
56         if xdata[i][j] != ":
57             xdata[i][j] = eval(xdata[i][j])
58         else:
59             xdata[i][j] = None
60 # print ('dataline',dataline)
61 # print ('xdata',xdata)
62 return xdata, numlines
63
64
65 def binary_summary_report(arr):
66     """
67     Takes a 2D binary array, sorts it, and prints a summary table.
68     """
69     nrows, ncols = arr.shape
70
71     # 1. Convert to Decimal using Dot Product (Fastest)
72     # Using 'object' dtype handles up to 100+ columns if necessary,
73     # but for < 64 bits, standard int64 is used automatically.
74     powers = 1 << np.arange(ncols)[::-1]
75     decimal_values = arr.dot(powers)
76
77     # 2. Get unique values and their frequencies
78     unique_vals, counts = np.unique(decimal_values, return_counts=True)
79
80     # 3. Reconstruct Binary Rows for display (Vectorized)
81     # This creates a bitmask to turn decimals back into [1, 0, 1, 0] format
82     unique_binary_rows = ((unique_vals[:, None] & (
83         1 << np.arange(ncols)[::-1])) > 0).astype(int)
84
85     # 4. Print the Summary Table
86     header = f"{'Unique Decimal':<15} | {
87         'Binary Equivalent':<25} | {'Count (Frequency)':<15}"
88     print(header)
89     print("-" * len(header))
90

```

```

91 for i in range(len(unique_vals)):
92     # Convert to list for cleaner printing
93     bin_row = unique_binary_rows[i].tolist()
94     print(f"{unique_vals[i]:<15} | {str(bin_row):<25} | {counts[i]:<15}")
95
96 # Return the dictionary as requested previously
97 return dict(zip(unique_vals, counts))
98
99
100 def draw_flexible_box_grid(n, label_list, datain, now):
101     total_boxes = 2**n
102
103     # 1. Determine Grid Dimensions (Width >= Height)
104     width = 2**((n + 1) // 2)
105     height = 2**(n // 2)
106
107     # 2. Handle Custom Labels
108     # Convert to list and pad with empty strings if the user provided too few
109     full_labels = list(label_list)
110     if len(full_labels) < total_boxes:
111         full_labels += [""] * (total_boxes - len(full_labels))
112
113     # Take only the amount needed for the grid
114     display_labels = full_labels[:total_boxes]
115
116     # 3. Reshape into the grid matrix
117     grid_labels = np.array(display_labels).reshape(height, width)
118
119     fig, ax = plt.subplots(figsize=(width * 1.5, height * 1.5))
120
121     for i in range(height):
122         for j in range(width):
123             # Coordinates for top-down numbering
124             rect = plt.Rectangle((j, height - i - 1), 1, 1,
125                                 fill=False, edgecolor='black', linewidth=2)
126             ax.add_patch(rect)
127
128             # Place the custom label
129             ax.text(j + 0.5, height - i - 0.5, grid_labels[i, j],
130                    ha='center', va='center',
131                    fontsize=12 if n < 5 else 9)
132
133     ax.set_xlim(0, width)
134     ax.set_ylim(0, height)
135     ax.set_aspect('equal')

```

```

136 ax.axis('off')
137 # plt.title(f"Custom Grid: n={n} ({width}x{height})", pad=20)
138 # newtitle = datain+ ' '+f"total_boxes={total_boxes}"+' '+now
139 newtitle = programname+' '+datain+' '+f"boxes={total_boxes}"+'\
140     ' '+titlenote+' '+now
141 plt.title(newtitle, pad=20)
142 plt.show()
143
144
145 programname='hidim_14.py'
146 # inout data
147 # cols separated by TAB
148 # reos sepsarated by RETURN
149 # floatihng point numers [0,1]
150 inputname = input("\ninput file namem (e.g. data6.txt)")
151 titlenote=input("\ninput title note (e.g. noise=0.7)")
152 data, ndata = getxn(inputname)
153 print('\ndata= ', data)
154 adata = np.array(data)
155 print('\nadata.shape (row,col)', adata.shape)
156 [nrow, ncol] = adata.shape
157 bdata = np.zeros((nrow, ncol), dtype=int)
158
159 # #make bdata[i,j] = 0 (if adata <= 0) = 1 (if adata > 0)
160 for irow in range(nrow):
161     for icol in range(ncol):
162         if adata[irow, icol] > 0.:
163             bdata[irow, icol] = 1
164 print('\nbdata\n', bdata)
165
166 # DO HAMMING DISTANCE COLS
167 target_0 = np.zeros((nrow), dtype=int)
168 target_1 = np.ones((nrow), dtype=int)
169 # for icol in range (ncol)
170 ham = np.zeros((ncol), dtype=int)
171 for icol in range(ncol):
172     col_test = bdata[:, icol].copy()
173     # print ('\ni, col_test= ',icol,col_test)
174     ham_0 = np.sum(col_test != target_0)
175     ham_1 = np.sum(col_test != target_1)
176     ham[icol] = min(ham_0, ham_1)
177     print('\ntarget_0, target_1, ham   ', ham_0, ham_1, ham[icol])
178
179 # first sort ham
180 # want the position in ham to be the min value

```

```

181 hamlow = np.zeros((ncol), dtype=int)
182 hamlow = np.argsort(ham)
183 cdata = bdata[:, hamlow] # pretty fancy, thank you Gemini
184 print('\nbdata\n', bdata)
185 print('\ncdata\n', cdata)
186
187 # THIS IS **TEST** OF ONLY <=63 COLS!!
188 # 1. Create a "Powers of 2" vector: [2^15, 2^14, ..., 2^0]
189 # Using bitwise shift (1 << n) is slightly faster than 2**n
190 powers = 1 << np.arange(ncol)[::-1]
191
192 # 2. Convert each row to a decimal integer using Matrix Multiplication
193 # This is the heavy lifting performed in optimized C/Fortran
194 decimal_values = cdata.dot(powers)
195
196 # 3. Get the sorted indices and reorder the original array
197 sorted_indices = np.argsort(decimal_values)
198 sorted_c = cdata[sorted_indices]
199 print('\nsorted_c\n', sorted_c)
200
201 # 4. Get Unique Decimal Values and their Counts
202 # return_counts=True makes this very fast
203 # unique_decimals=np.zeros((32), dtype=int)
204 unique_decimals, counts = np.unique(decimal_values, return_counts=True)
205 # labels=unique_decimals.astype(str).tolist()
206
207 # 5. Result Display
208 print("Sorted Array:\n", sorted_c)
209 print("\nUnique Decimal Values:", unique_decimals)
210 print("Frequency Counts:  ", counts)
211
212 # --- Example Usage ---
213 # Creating a dummy array with 100 rows and 5 columns
214 # data = np.random.randint(0, 2, size=(100, 5))
215 results = binary_summary_report(cdata)
216
217 # OK that's ALL sorted, let's now do the BIGLOOP
218 # OK MESSED UP dont need to run SORT but do need COURNTS
219 now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
220 for bigloop in range(6): # not doing anything yet
221     n = 2**bigloop
222     edata = sorted_c[:, 0:bigloop]
223     mrow, mcol = edata.shape
224     mpowers = 1 << np.arange(mcol)[::-1]
225     mdecimal_values = edata.dot(mpowers)

```

```
226 sorted_indices = np.argsort(mdecimal_values)
227 sorted_e = edata[sorted_indices]
228 munique_decimals, mcounts = np.unique(mdecimal_values, return_counts=True)
229 # 5. Result Display
230 print("Sorted Array:\n", sorted_e)
231 print("\nUnique Decimal Values:", munique_decimals)
232 print("Frequency Counts:  ", mcounts)
233 results = binary_summary_report(edata)
234 # my_labels = ["Start", "A1", "B2", "C3", "D4", "E5", "F6", "End"]
235 my_labels = mcounts.astype(str).tolist()
236 draw_flexible_box_grid(bigloop, my_labels, inputname, now)
```